Innovation Centre for Education

YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project –Low Level Design

on

AI-Driven Self-Healing Cloud for Healthcare.

Student Name: ASHIYANA HIBA

Industry Mentor: **Guided by:**

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience.... 6
   1.3. System overview.... 6

2. System Design. 6
   2.1. Application Design.... 6
   2.2. Process Flow.... 6
   2.3. Information Flow.... 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8

3. Data Design.... 8
   3.1. Data Model.... 8
   3.2. Data Access Mechanism.... 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces.... 9

5. State and Session Management.... 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects.... 10
   7.2. Performance Aspects.... 10

8. References.... 11

1. Introduction

1.1 Scope of the document

This Low-Level Design (LLD) document provides an in-depth technical description of the internal working of the project titled **"AI-Driven Self-Healing Cloud for Healthcare."** It focuses on the detailed implementation of modules, internal logic, data structures, and execution flow of the system.

The scope of this document includes:

- Detailed explanation of system modules and their interactions
- Function-level implementation and logic breakdown
- Data handling and temporary storage mechanisms
- Integration of monitoring, self-healing, and visualization components
- Execution workflow and error handling mechanisms
- This document complements the High-Level Design by providing deeper insights into how each component is implemented at the code level.

1.2 Intended Audience

This document is intended for:

- Developers responsible for implementing or modifying the system
- Students studying system design, DevOps, and automation
- Faculty members evaluating the technical depth of the project
- Beginners interested in understanding process monitoring systems

1.3 System Overview

The system is designed as a real-time monitoring and recovery application that continuously observes system processes and ensures their availability through automated recovery.

At the core, the system:

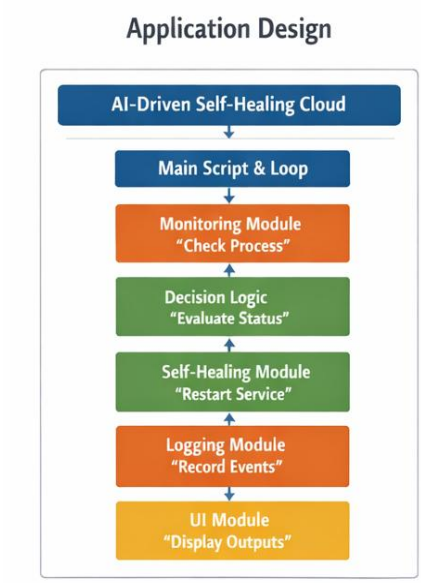
- Monitors a specific process (e.g., Notepad)
- Detects whether the process is running
- Automatically restarts the process if it stops

- Logs all activities with timestamps
- Displays system status and performance metrics in a dashboard

The implementation is lightweight and uses Python libraries such as os, psutil, and datetime, along with Streamlit for UI.

2. System Design

2.1 Application Design



At the low level, the application is structured as a continuous execution system driven by a loop. The system integrates multiple modules that interact through function calls.

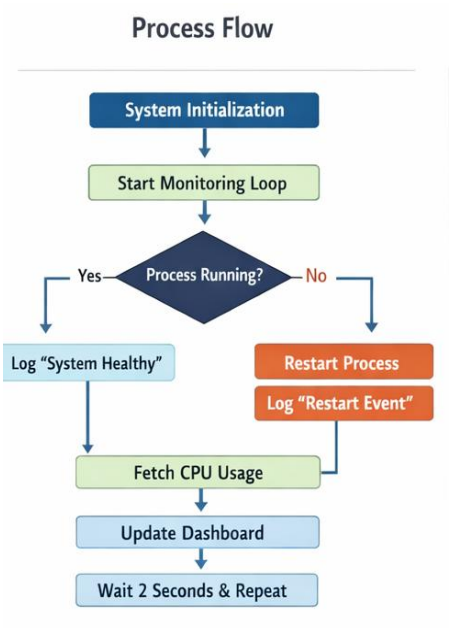
Internal Working:

- The main script initializes session variables
- A loop runs continuously at fixed intervals
- Each iteration checks process status
- Based on the result, appropriate action is taken
- UI is updated dynamically

Modules Interaction:

- Monitoring module → checks process
- Decision logic → evaluates status
- Self-healing module → executes restart
- Logging module → records events
- UI module → displays outputs

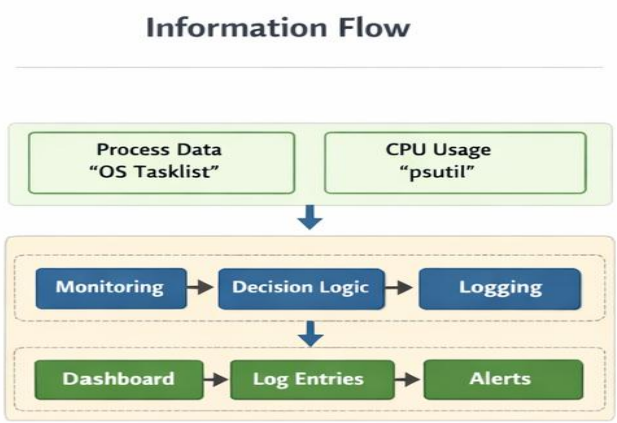
2.2 Process Flow



- System initialization begins
- Session variables (logs, CPU data) are initialized
- Monitoring loop starts
- System fetches process list using OS command
- Process name is matched with running processes
- If process is found:
- Log "System Healthy"
- If process is not found:

- Trigger restart command
- Log restart event
- Fetch CPU usage using psutil
- Append CPU value to data list
- Update Streamlit dashboard
- Wait for specified delay (e.g., 2 seconds)
- Loop continues indefinitely

2.3 Information Flow



Input Layer:

- Process data obtained from OS (tasklist)
- CPU usage obtained from psutil

Processing Layer:

- String matching algorithm for process detection
- Conditional statements for decision making
- Execution of system commands for restart

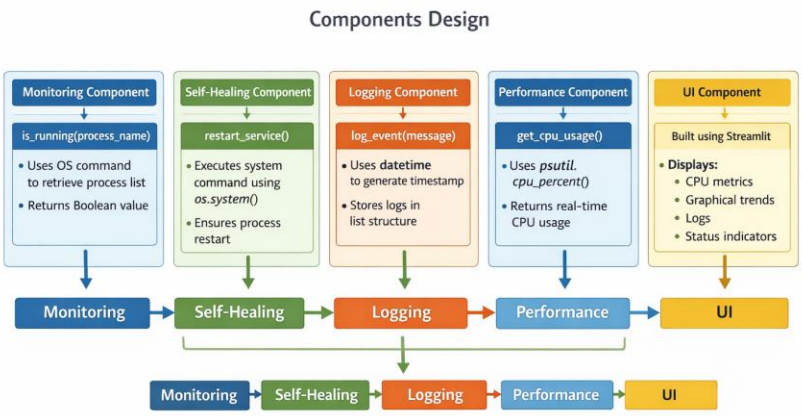
Output Layer:

- Dashboard updates (metrics, graphs)
- Log entries with timestamps
- Alert messages (failure notifications)

Data Movement:

Input → Monitoring → Decision → Action → Logging → UI

2.4 Components Design



Monitoring Component

- Function: is_running(process_name)
- Uses OS command to retrieve process list

- Converts output to lowercase for case-insensitive comparison
- Returns Boolean value

Self-Healing Component

- Function: restart_service()
- Executes system command using os.system()
- Ensures process restart

Logging Component

- Function: log_event(message)
- Uses datetime to generate timestamp
- Stores logs in list structure

Performance Component

- Function: get_cpu_usage()
- Uses psutil.cpu_percent()
- Returns real-time CPU usage

UI Component

- Built using Streamlit
- Displays:
    - CPU metrics
    - Graphical trends
    - Logs
    - Status indicators

2.5 Key Design Considerations

- **Modularity:** Each function performs a single task
- **Efficiency:** Minimal resource consumption
- **Reliability:** Continuous monitoring loop
- **Extensibility:** Can support multiple processes
- **User Experience:** Clean and interactive UI

2.6 API Catalogue

Function	Description	Input	Output
is_running()	Checks process status	Process name	Boolean
restart_service()	Restarts application	None	None
log_event()	Logs event	String	Log string
get_cpu_usage()	CPU usage	None	Float

3. Data Design

3.1 Data Model

The system uses in-memory data structures:

- **Logs:** List of strings
- **CPU Data:** List of float values
- **Status Flag:** Boolean

Example:

logs = []

cpu_data = [] running = True

3.2 Data Access Mechanism

Data is accessed using:

- OS library → process information
- psutil → system performance
- Streamlit session state → data persistence

3.3 Data Retention Policies

- Logs stored temporarily
- Only recent logs displayed
- CPU data limited to last 30 entries

3.4 Data Migration

Not applicable due to lack of database. Future implementation may include cloud storage.

4. Interfaces

The system provides a web-based interface using Streamlit.

UI Features:

- Dashboard layout
- CPU usage graph
- Logs panel
- Start/Stop buttons
- Status indicators

5. State and Session Management

The system uses Streamlit session state to manage:

- Monitoring status (running/stopped)
- Log data

- CPU usage history

This ensures continuity during execution and prevents data loss during UI refresh.

6. Caching

Caching is implemented using Streamlit mechanisms to:

- Store recent CPU usage values
- Avoid redundant computations
- Improve performance

This ensures smooth and efficient dashboard updates.

7. Non-Functional Requirements

7.1 Security Aspects

- No sensitive data is processed
- Local system execution ensures safety
- Can be extended with authentication in future

7.2 Performance Aspects

- Lightweight Python script
- Minimal CPU usage
- Real-time monitoring with low latency
- Efficient loop execution

8. References

1. Python Official Documentation – https://docs.python.org
2. Streamlit Documentation – https://docs.streamlit.io
3. psutil Library Documentation – https://psutil.readthedocs.io
4. Microsoft Task Scheduler Guide
5. Cloud Computing Concepts – Self-Healing Systems